

# E-Commerce Revenue & Customer Retention Analytics Platform

## Project details · Skills to learn · Step-by-step implementation

A practical portfolio project for a fresher data analyst resume. Build a reliable analytics pipeline, explain the business results, and package the work so another person can reproduce it.

| PROJECT AT A GLANCE | PLAN                                                                                                |
|---------------------|-----------------------------------------------------------------------------------------------------|
| Dataset             | Olist public Brazilian e-commerce data; approximately 100,000 orders from 2016–2018 [1].            |
| Core stack          | Python / pandas · PostgreSQL / SQL · Power BI / DAX · Git / GitHub                                  |
| Core analyses       | Sales, repeat purchasing, RFM, cohorts, delivery performance and customer reviews                   |
| Final outputs       | Reproducible code, validated data model, five-page dashboard, executive summary and recommendations |
| Suggested effort    | 60–90 focused hours if basics are familiar; allow extra time for learning.                          |

Prepared for Paras · 6 October 2026

## How to use this guide

Read the project brief first. Learn each concept just before the phase that uses it. Complete the phase check before starting the next phase. All business targets and impact examples in this guide are proposed scenarios, not findings from an executed analysis.

# 01 / Project brief

## The business situation

Use a fictional Growth & Operations team as the stakeholder for analysis of real historical Olist data. Management wants to understand which categories and customers drive sales, where repeat purchasing is weak, and which delivery issues are associated with poor reviews.

| STAKEHOLDER        | QUESTION                                                | DECISION                                                |
|--------------------|---------------------------------------------------------|---------------------------------------------------------|
| Growth manager     | Which customers are likely to purchase again?           | Prioritize a retention campaign and define a pilot.     |
| Operations manager | Which sellers or regions have frequent late deliveries? | Investigate fulfillment problems and track improvement. |
| Category manager   | Which categories contribute the most sales?             | Choose categories for further commercial investigation. |

## Primary objective and success metrics

Produce evidence that helps management choose one retention pilot and one operational improvement. Primary customer KPI: **90-day repeat purchase rate** among customers with a complete 90-day observation window. Operational KPI: **late delivery rate** among orders with valid actual and promised delivery dates.

Compute baselines from the data first. Example proposed targets: increase 90-day repeat rate by 1 percentage point, or reduce late delivery rate by 2 percentage points in a future pilot. These are planning targets and require real follow-up data to verify.

## What you will build

- A raw → staging → analytics pipeline with documented cleaning and reconciliation checks.
- SQL analysis and Python notebooks that answer business questions with charts and uncertainty.
- A BI model with separate facts for different row grains and clear KPI definitions.
- Five dashboard pages with filters, business insights and consistent totals.
- An opportunity table and executive summary linking evidence to actions and scenario estimates.

## Scope boundaries

Olist does not provide the complete costs needed for profit, browsing events needed for a conversion funnel, or a randomized campaign needed for real A/B testing. Do not invent those fields. Customer inactivity is an observed pattern, not confirmed churn. Keep predictive modeling optional.

### Completion check

Write docs/business\_requirements.md with stakeholder, questions, KPI definitions, proposed targets and limitations. Record actual baselines only after analysis.

## 02 / Technologies to learn

You do not need to master every tool before starting. Aim to complete the practical exercise in each row.

| TOOL                 | WHAT TO KNOW                                                                                   | READINESS EXERCISE                                                             |
|----------------------|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Python + pandas      | Functions, files, environments, DataFrames, dtypes, missing values, merge, groupby and dates.  | Read orders and items; aggregate sales by order; validate a many-to-one merge. |
| PostgreSQL + SQL     | Tables, schemas, keys, numeric/date types, joins, CTEs, windows, views and indexes.            | Create two tables, load CSV data, join them and compute monthly totals.        |
| SQLAlchemy + psycopg | Database connections, parameterized operations, bulk loading and transactions.                 | Load a small table from a script without exposing a password.                  |
| Jupyter + plotting   | Re-runnable notebooks; matplotlib or seaborn; readable tables and charts.                      | Plot a monthly trend and write a two-sentence interpretation.                  |
| SciPy + statistics   | Distributions, effect sizes, confidence intervals, tests and assumptions.                      | Compare two groups and explain uncertainty without a causal claim.             |
| Power BI             | Power Query, relationships, star schema, DAX measures, filter context and visual interactions. | Create a date relationship and an AOV measure; test a state slicer.            |
| Git + GitHub         | Commits, .gitignore, README, branches and source attribution.                                  | Commit scripts and instructions while excluding data and credentials.          |

### Your Mac setup decision

Power BI Desktop requires Windows [2]. You can do Python, PostgreSQL and SQL work on your Mac, then use an accessible Windows computer or Windows environment for the BI phase. If Windows is unavailable, use Tableau as the dashboard alternative; replace DAX learning with Tableau calculations. Confirm the BI route before investing time in dashboard design.

### Suggested learning order

Python basics → pandas joins and dates → SQL joins and aggregation → database loading → data quality → fact/dimension modeling → SQL windows → RFM and cohorts → statistics → Power BI and DAX. Learn advanced forecasting or ML after the core project works.

## 03 / Concepts you must understand

| CONCEPT               | MEANING IN THIS PROJECT                                                                                |
|-----------------------|--------------------------------------------------------------------------------------------------------|
| Table grain           | What one row represents. An order row, an item row and a payment row are different units.              |
| Primary / foreign key | A primary key identifies a row; a foreign key links it to another table. Some keys are composite.      |
| Join cardinality      | One-to-many joins multiply rows. Two item rows joined to two payment rows create four rows.            |
| ETL and layers        | Extract CSVs; transform into typed, validated staging tables; load analytical facts and dimensions.    |
| Fact / dimension      | Facts store events and measurements. Dimensions store attributes used to filter and group events.      |
| Metric denominator    | Specify which records qualify. Late rate must exclude orders without valid delivery evidence.          |
| Observation window    | Recent customers have less time to repurchase. Compare customers with equal follow-up.                 |
| RFM / cohort          | RFM summarizes purchase behavior; cohorts track customers first observed purchasing in the same month. |
| Statistical evidence  | Effect size describes magnitude; a confidence interval describes uncertainty; a p-value is not impact. |
| Reproducibility       | A fresh run produces the same outputs using the recorded files, rules, dates and dependencies.         |

### Three mistakes to prevent from day one

- **Repeated money:** never sum order payment totals after attaching them to every order item. Aggregate each child table at its intended grain before joining.
- **Wrong customer identifier:** link orders using customer\_id, but track people and repeat purchases using customer\_unique\_id [1].
- **Incorrect interpretation:** item sales are merchandise value, not the marketplace's accounting revenue or profit. Low repeat activity is not proof that a customer has permanently left.

### Small practice before the real dataset

Create one order with two items priced 100 and 50 and two payment records of 90 and 60. An unsafe item/payment join gives four rows and doubles totals. Your correct order summary must show merchandise value 150 and payments 150. Use this example to explain grain in interviews.

#### Completion check

You can explain why row multiplication occurs, name each table's grain, and calculate a percentage with its correct denominator.

## 04 / Architecture and setup

| LAYER            | INPUT → OUTPUT                            | RESPONSIBILITY                                             |
|------------------|-------------------------------------------|------------------------------------------------------------|
| Raw              | Original CSVs → raw tables                | Keep source values and record file hashes / row counts.    |
| Staging          | Raw → typed tables + quality flags        | Apply deterministic transformations and record exceptions. |
| Analytics        | Staging → facts, dimensions and KPI views | Expose stable grains and approved metric logic.            |
| Analysis         | Analytics → SQL results + notebooks       | Investigate sales, customers, retention and operations.    |
| BI and reporting | Analytics → dashboard + executive summary | Make results useful for stakeholder decisions.             |

### Set up in this order

- Create a Git repository and folders for src, sql, notebooks, docs, reports and powerbi.
- Install Python, PostgreSQL and an SQL client. Create the ecommerce\_analytics database and raw, staging and analytics schemas.
- Create a Python environment. Install pandas, numpy, sqlalchemy, psycopg, python-dotenv, jupyter, matplotlib, seaborn and scipy. Record tested versions.
- Use a local .env for the database connection. Commit an .env.example containing placeholders; ignore .env and raw data.
- Download the dataset, record its source and license, and keep the original files unchanged.

```
python3 -m venv .venv
source .venv/bin/activate # macOS / Linux
# Windows: .venv\Scripts\activate
python -m pip install pandas numpy sqlalchemy "psycopg[binary]" \
    python-dotenv jupyter matplotlib seaborn scipy
python -m pip freeze > requirements.txt
```

### Design a simple execution interface

Implement scripts such as src/ingest.py, src/build\_staging.py, src/build\_marts.py and src/validate.py. Each should take configuration from environment variables or a config file, log its inputs and outputs, and return a failure status for critical quality errors. Notebook execution is for analysis; core transformations belong in scripts.

#### Completion check

An isolated environment works, the database connection succeeds, and a small CSV can be loaded. Keep the credentials out of Git history.

## 05 / Understand and ingest the data

The dataset covers historical Brazilian marketplace orders, not current company performance [1]. Keep monetary results in **BRL (R\$)**. Profile the downloaded files to establish actual row counts and coverage; do not copy illustrative counts from a plan.

| FILE / TABLE         | EXPECTED GRAIN / KEY                          | IMPORTANT HANDLING                                   |
|----------------------|-----------------------------------------------|------------------------------------------------------|
| orders               | One order / order_id                          | Status and lifecycle timestamps; profile uniqueness. |
| customers            | Customer record / customer_id                 | customer_unique_id links repeated purchases.         |
| order_items          | One order item / order_id + order_item_id     | Item price, freight, product and seller.             |
| payments             | Payment entry / order_id + payment_sequential | An order may have several payments.                  |
| reviews              | Review entry; validate candidate keys         | Profile repeated IDs and multiple records per order. |
| products             | Product / product_id                          | Category and physical attributes.                    |
| sellers              | Seller / seller_id                            | Seller location; not a customer attribute.           |
| geolocation          | Location observation; ZIP prefix repeats      | Reduce to one ZIP-prefix record before lookup joins. |
| category_translation | Category translation / category name          | Left join so untranslated categories remain visible. |

### Build the ingestion script

- Read identifiers and ZIP prefixes as strings so formatting is preserved. Keep raw date values for audit; parse them in staging.
- Write a source manifest: filename, SHA-256 hash, row count, column names, ingestion time and dataset version / download date.
- Load raw tables reproducibly. A full rebuild is sufficient for this static dataset; use database transactions where practical.
- Record null counts, duplicate counts and earliest/latest timestamps before cleaning. Keep a list of rejected parsing values.

### Choose the reporting cutoff

Select the last reliably complete purchase period after inspecting the timeline. Use a fixed cutoff in configuration. End-of-file orders may not have had time to complete delivery. Recent cohorts may not have a full 90 days of follow-up. Label and exclude incomplete periods only from metrics that require maturity.

#### Completion check

All nine source files are accounted for, raw row counts reconcile, and docs/data\_dictionary.md explains keys and relationships.

## 06 / Cleaning and validation

Treat quality checks as decisions with evidence. A null value is not automatically an error; an undelivered order can legitimately have no delivery timestamp.

| CHECK               | RULE / ACTION                                                                                                                     |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Duplicate keys      | Fail on duplicate order_id. Profile item/payment composite keys. Remove only proven exact duplicates and log counts.              |
| Missing values      | Retain lifecycle nulls with flags. Use Unknown for missing categories; do not invent a delivery date or rating.                   |
| Types and ranges    | Parse timestamps explicitly; use NUMERIC for money in SQL. Flag negative amounts and ratings outside 1–5.                         |
| Time consistency    | Flag delivery before purchase, approval anomalies and impossible durations. Keep flagged rows available for investigation.        |
| Relationships       | Count orphan items, missing products/sellers and unmatched translations. Quarantine critical key failures.                        |
| Extreme values      | Inspect high prices/freight. Preserve plausible large orders; report sensitivity rather than removing all outliers.               |
| Review multiplicity | Keep original reviews. For an order-level score, choose the latest valid entry by documented timestamp and stable tie-break rule. |
| Geolocation         | Profile coordinate ranges; use a reproducible median coordinate per ZIP prefix. Record spread and unmatched prefixes.             |

### Create a quality ledger

For each rule, record: table, rule name, rows checked, rows flagged, action, rows removed, reason and run ID. Report before/after row counts and null rates. Distinguish excluded metric observations from deleted records.

### Minimum automated checks

- Primary-key uniqueness; required identifier nulls; foreign-key coverage and valid score ranges.
- Merchandise totals reconcile from staging items to item facts and order aggregates.
- Payment totals reconcile independently; differences from price + freight are reported, not silently forced to match.
- After dimension joins, row counts and monetary totals are unchanged.
- A rerun produces the same analytics outputs without duplicate inserts.

#### Completion check

reports/data\_quality\_report.md includes measured counts, transformation decisions and unresolved exceptions. Critical key or reconciliation failures block the next phase.

## 07 / Build the analytical data model

Use a small constellation of facts sharing dimensions. Avoid a single FactOrders table that mixes item-level product data with repeated order-level payments and ratings. A consistent fact grain is fundamental to a reliable BI model [3].

| ANALYTICS TABLE          | ONE ROW REPRESENTS        | MEASURES / KEYS                                                                                                                                    |
|--------------------------|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| fact_order               | One order                 | order_id, customer_key, purchase_date_key, destination_key, status, order GMV, freight, validity flags, delivery days, late flag, selected review. |
| fact_order_item          | One item within an order  | order_id + item_id, customer_key, product_key, seller_key, purchase_date_key, destination_key, price, freight.                                     |
| fact_payment             | One payment entry         | order_id + sequence, customer_key, purchase_date_key, payment type, amount, installments.                                                          |
| dim_customer             | One customer_unique_id    | Stable customer identity. Retain the raw customer_id mapping in staging.                                                                           |
| dim_product / dim_seller | One product / seller      | Category and product attributes; seller location.                                                                                                  |
| dim_date / dim_location  | One day / destination key | Calendar attributes; delivery address city/state/ZIP prefix.                                                                                       |

### Implementation sequence

- Create dimension tables and deterministic keys. Store destination at the order grain because a person may order to different addresses.
- Aggregate items once per order to derive order GMV and freight. Aggregate payments independently, and select one review using the written rule.
- Join these order aggregates to orders using one-to-one or many-to-one checks. Build item and payment facts separately.
- Add a continuous calendar table. Use purchase date as the active BI relationship; use role-specific date tables or an inactive relationship for delivery-date views.
- Connect dimensions to facts with one-to-many, single-direction relationships. Keep fact tables unconnected directly in the BI model.

### Seller attribution needs care

An order may contain items from several sellers. Seller GMV belongs to the item fact. An order-level delivery date or review cannot establish one seller's responsibility. If assigning order lateness to participating sellers, label the metric as associated order lateness and avoid summing overlapping order counts across sellers.

#### Completion check

Publish a model diagram and table-grain dictionary. Check dimension keys are unique and item/order totals agree at the same scope.

## 08 / Define KPIs before charting

Write docs/KPI\_dictionary.md with formula, source grain, qualifying statuses, date basis, denominator, exclusions and currency. Recommended core scope: delivered orders for merchandise and repeat-purchase reporting; provide an all-status order breakdown separately.

| KPI                       | DEFINITION / CAVEAT                                                                                                                                        |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Merchandise value (GMV)   | Sum item price for qualifying delivered orders. Report BRL. This is the project's sales proxy, not platform revenue.                                       |
| Order count / AOV         | Count unique qualifying order_id. AOV = GMV / that order count, with the same filters.                                                                     |
| Observed customers        | Distinct customer_unique_id in the qualifying scope. Customer records are not the same as unique people.                                                   |
| Overall repeat rate       | Customers with 2+ qualifying orders / customers with 1+ within a fixed window. Label as descriptive window-based rate.                                     |
| 90-day repeat rate        | Among first-observed buyers with full 90-day follow-up: share with another purchase within 90 days. Define eligibility by purchase time and status policy. |
| Cohort activity retention | Active customers in month N / first-purchase cohort size. Unobserved future cells are blank, not zero.                                                     |
| Late delivery rate        | Orders delivered after the promised calendar date / delivered orders with valid actual and promised dates.                                                 |
| Delivery days             | Actual delivery timestamp minus purchase timestamp, converted to days; exclude invalid chronology.                                                         |
| Average review / coverage | Average selected valid order rating. Coverage = rated qualifying orders / qualifying orders. Unrated is not score zero.                                    |
| MoM GMV growth            | (Current full-month GMV – previous full-month GMV) / previous full-month GMV. Return blank for a missing/zero comparison.                                  |

### Consistency checks across SQL and BI

Choose one complete month, one state and one category. Compare the relevant SQL results against dashboard outputs. State filters can affect both order and item metrics. Category and seller filters naturally affect item facts; order-level KPIs need an explicitly derived order cohort or must not react to those slicers. Document the chosen behavior.

#### Completion check

Every dashboard KPI has an approved definition, and its filter behavior is explicit. No chart combines mismatched populations or grains.

## 09 / Implement SQL analysis

| MODULE               | QUESTIONS / SKILLS TO SHOW                                                                                                     |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------|
| 01_validation.sql    | Uniqueness, orphan keys, null counts and totals reconciliation.                                                                |
| 02_sales.sql         | Monthly GMV, AOV, category/state contribution and growth. GROUP BY, CASE, LAG and CTEs.                                        |
| 03_customers.sql     | Unique buyers, first purchase, repeat frequency and inter-purchase gaps. MIN, COUNT DISTINCT and LEAD.                         |
| 04_operations.sql    | Delivery distributions, late rate, review coverage and seller-associated metrics. Conditional aggregation and minimum volumes. |
| 05_rfm.sql           | Customer recency, frequency and monetary value at a fixed cutoff. Explicit scoring rules and segment CASE.                     |
| 06_cohorts.sql       | First-purchase cohorts, month offsets and maturity flags. CTEs and calendar joins.                                             |
| 07_opportunities.sql | Eligible audiences, seller/category exceptions and ranking. Views, thresholds and prioritization.                              |

### Example: monthly merchandise value and growth

Assumes analytics.fact\_order has one row per order and order\_gmv is already reconciled. Configure the cutoff so partial months do not enter this comparison.

```
WITH monthly AS (
  SELECT date_trunc('month', purchase_ts)::date AS month,
         SUM(order_gmv) AS gmv
  FROM analytics.fact_order
  WHERE status = 'delivered'
        AND purchase_ts >= :start_date
        AND purchase_ts < :end_exclusive
  GROUP BY 1
), compared AS (
  SELECT *, LAG(gmv) OVER (ORDER BY month) AS prev_gmv
  FROM monthly
)
SELECT month, gmv,
       100.0 * (gmv - prev_gmv) / NULLIF(prev_gmv, 0) AS growth_pct
FROM compared ORDER BY month;
```

Named parameters above assume SQLAlchemy execution. When testing in an SQL client, replace them with explicit date literals. For a missing month, join to a calendar-month spine before computing LAG; otherwise it compares with the previous observed month.

### Show practical optimization

Use EXPLAIN ANALYZE on one slow query. Compare a plan before and after an appropriate index on join/filter columns, or after pre-aggregation. Record execution conditions and results; do not claim that every index improves every query. Window functions are documented in [4].

#### Completion check

Queries run in a documented order, all major KPIs reconcile, and at least one joins/CTE/windows example is explained in the README.

# 10 / Customer, RFM and cohorts

## RFM: build a repeatable customer snapshot

- Set a fixed historical snapshot date just after the reporting cutoff. Do not use today for a dataset ending years ago.
- For each customer\_unique\_id, compute Recency = days since latest qualifying purchase; Frequency = distinct orders; Monetary = sum merchandise value.
- Inspect distributions first. Many customers may have just one order, so five quantile bins can be invalid or misleading.
- Use documented recency / monetary thresholds or quantiles with ties preserved. Define frequency bands from observed counts; do not split equal-frequency customers arbitrarily.
- Create segments with explicit rules, such as recent high-spend buyers, recent repeat buyers, and inactive high-spend buyers. Call inactivity a targeting rule, not confirmed churn.

## Cohort retention: compare like with like

- Assign each customer to their first observed qualifying purchase month. It is first observed purchase, not necessarily their first-ever purchase.
- Count distinct active customers by cohort and months since that first purchase. Divide by the original cohort size.
- Use one complete-month cutoff. Blank future/incomplete cells and show cohort sample sizes. Month 0 is 100% by construction.
- Compare cohorts at the same age: Month 1 against Month 1, not a young cohort against the full history of an older one.
- Create the separate 90-day repeat metric only for customers with sufficient follow-up. Track exclusions and the status policy.

## Customer value: keep the claim defensible

Report observed customer spend, repeat-order GMV, and 90-day customer value for mature cohorts. These are useful historical value proxies. Do not label observed spend as predicted lifetime value. A future CLV model would need explicit horizon, retention assumptions and validation beyond the core scope.

## Outputs to create

analytics.customer\_rfm\_snapshot, analytics.cohort\_retention and analytics.customer\_90d\_summary; a segment distribution chart; retention heatmap; and a short explanation of how the observation window changes the results.

### Completion check

Tied RFM values are handled consistently, each person is counted once per cohort-month, and the heatmap distinguishes zero activity from unobserved months.

# 11 / Statistics and opportunity analysis

## Question: are late deliveries associated with lower ratings?

- Use one selected review per order and valid delivery evidence. Show group sizes, missing-review coverage, score distributions and low-rating shares.
- Define H0 as equal rating distributions for late and on-time orders; define H1 as a difference. A two-sided Mann-Whitney U test is a reasonable starting point for ordinal ratings [6].
- Report the p-value alongside a rank-based effect size and a bootstrap confidence interval for the difference in low-rating shares (ratings 1-2). Set the random seed.
- Do not interpret Mann-Whitney as purely a median test without additional distribution assumptions. Account for repeated customers, for example with a customer-level bootstrap or a one-order-per-customer sensitivity check.
- Compare results by category, state and period. Observational associations can reflect confounding and review selection; they do not prove that lateness caused the score difference.

## Business Opportunity Finder

| OPPORTUNITY                       | EVIDENCE / NEXT ACTION                                                                                                                   |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Inactive high-spend buyers        | Historical value + recency + eligibility rules. Propose a consented retention pilot; Olist IDs are anonymized and not contactable leads. |
| High-GMV seller-associated delays | Order volume, late share and confidence interval. Investigate fulfillment; avoid assigning order-level blame to one seller.              |
| Freight-heavy categories          | Freight / merchandise ratio and valid order counts. Investigate pricing / logistics; freight collected is not measured shipping cost.    |
| Strong categories with low repeat | Sales and maturity-adjusted repeat patterns. Investigate purchase cadence before proposing a campaign.                                   |

Export opportunity\_id, entity/segment, evidence, sample size, period, historical GMV exposure, priority rule, proposed owner and next action. Label historical exposure clearly; it is not guaranteed future revenue at risk. Use an explicit minimum-volume threshold and test its sensitivity.

## Optional extension: late-delivery prediction

Only after the core deliverables: define when a prediction is made; use information available then; split train/test chronologically; build historical seller features using earlier orders only. Exclude actual delivery dates and post-delivery reviews. Compare a simple baseline with logistic regression using precision, recall, PR-AUC and calibration at a business threshold.

### Completion check

The statistical notebook reports magnitude, uncertainty, selection limits and assumptions. Opportunity priorities can be traced back to measured evidence.

# 12 / Build the Power BI dashboard

Connect Power BI to the analytics schema using Import for the initial build. Load facts and dimensions; set types and relationships; mark the calendar as a date table. Keep core business logic in documented SQL and DAX, with Power Query for import shaping [3].

| PAGE                         | QUESTION / VISUALS                                                                                                                    |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| 1. Executive overview        | How is the business performing? GMV, delivered orders, AOV, customers, 90-day repeat and late rate; monthly trend and top categories. |
| 2. Sales and products        | Where does merchandise value come from? Category/state trends, item/seller contribution and freight ratio.                            |
| 3. Customers                 | Who buys and who is valuable? Observed buyers, first-time vs repeat activity, RFM segments and spend distribution.                    |
| 4. Retention                 | Are cohorts purchasing again? Cohort heatmap, mature-cohort 90-day repeat, sample sizes and historical customer value.                |
| 5. Operations and experience | Where are delivery problems? Late rate, delivery days, region trends, seller-associated lateness and rating distributions / coverage. |

## Starter DAX measures

```

GMV :=
CALCULATE(
    SUM(fact_order[order_gmv]),
    fact_order[status] = "delivered"
)

Delivered Orders :=
CALCULATE(
    COUNTROWS(fact_order),
    fact_order[status] = "delivered"
)

AOV := DIVIDE([GMV], [Delivered Orders])

```

These measures use the unique order fact and shared date/location filters. Category and seller belong to item-level measures. For an order KPI filtered by category, define the qualifying set of order IDs explicitly and apply it to the order fact; never let an accidental relationship decide the denominator.

## Make the report usable and verify it

- Include date/state filters; only enable category/seller filters for measures with defined behavior. Add drill-through for entity details where useful.
- Use clear units, BRL formatting, period labels, sample sizes and definitions in tooltips. Show a concise evidence-based insight per page.
- Compare cards with SQL under at least three representative filter combinations. Test empty data, missing previous periods, cross-highlighting and reset filters.

### Completion check

Five pages answer five decisions. Totals reconcile to SQL, and screenshots / a report PDF make the work reviewable without a live BI link.

# 13 / Recommendations and impact

## Turn findings into actions

Write 5–8 measured insights and 3–5 recommendations after running the analysis. Every recommendation needs: finding, period and population, evidence, caveat, owner, action, proposed deadline, scenario estimate and monitoring KPI. Avoid recommendations that the dataset cannot support.

| FIELD                | EXAMPLE STRUCTURE — FILL WITH REAL RESULTS                                                               |
|----------------------|----------------------------------------------------------------------------------------------------------|
| Finding and evidence | Seller-associated late rate exceeds the comparison group; show counts, period and interval.              |
| Action and owner     | Operations investigates fulfillment for the eligible high-volume seller group.                           |
| Timing               | Propose a 30-day investigation followed by a measured pilot; this is a plan.                             |
| Monitoring           | Late rate, low-rating share, review coverage and eligible order volume.                                  |
| Validation           | Compare with a similar control group or randomized rollout if future operational data becomes available. |

## Estimate an opportunity as a scenario

**Formula:** incremental merchandise value = eligible customers × incremental purchase probability × expected qualifying order value. Use incremental response above a baseline, not total campaign response. Estimate campaign cost separately; absent margins, do not call the result profit or ROI.

| ILLUSTRATIVE INPUT               | CONSERVATIVE | BASE     | OPTIMISTIC |
|----------------------------------|--------------|----------|------------|
| Eligible audience                | 1,000        | 1,000    | 1,000      |
| Incremental purchase probability | 1%           | 3%       | 5%         |
| Expected order GMV               | R\$150       | R\$150   | R\$150     |
| Incremental GMV                  | R\$1,500     | R\$4,500 | R\$7,500   |

These inputs are invented only to demonstrate the calculation. Replace audience and order value with measured inputs; keep response assumptions clearly labeled. Report results as scenario-based estimates. A historical dataset cannot verify realized uplift from a campaign you have not run.

## Executive summary: one page

Include the business question, data scope, three leading findings with numbers, three recommended actions, a scenario estimate and limitations. Keep charts minimal and language understandable to a growth or operations manager. An optional 5–8 slide deck can use the same story.

### Completion check

Each action traces to actual evidence. Estimated impact is labeled as hypothetical, and no fictional finding appears in the README or resume.

# 14 / Repository and reproducibility

```
ecommerce-analytics-platform/
  README.md           # problem, results, setup, limits
  requirements.txt     # tested dependency versions
  .env.example        # placeholders only
  config/analysis.yml # cutoff, thresholds, seed
  src/               # ingest, transform, validate
  sql/              # schema, marts, analyses
  notebooks/        # EDA, cohorts, statistics
  data/README.md     # download + license instructions
  docs/             # KPI dictionary, grains, model
  powerbi/          # report + measures documentation
  assets/           # screenshots and diagrams
  reports/          # quality report, summary, actions
```

## Document the exact run sequence

- Download the original dataset from the cited source; verify files against the manifest.
- Create the Python environment, install recorded dependencies and configure the database connection.
- Run schema creation → ingestion → staging → facts/dimensions → validations → SQL analyses.
- Run notebooks from a fresh kernel against the generated analytics layer. Save charts and a run manifest.
- Refresh Power BI, verify selected KPI totals, and export dashboard screenshots and the executive summary.

## README contents

Business question; dataset and license; scope and cutoff; architecture; fact grains; cleaning decisions; KPI definitions; SQL/Python methods; dashboard previews; measured findings; recommendations; scenario estimates; limitations; setup and run instructions. State exactly what is implemented and what remains optional.

## Governance and maintenance notes

Use a database account with only the permissions the pipeline needs; use read-only access for BI where practical. Keep secrets out of code and logs. Minimize review text in public screenshots. This is a static historical dataset, so refresh is on demand; record how a future monthly ingestion would rerun quality checks. Publish dataset source/license instructions rather than redistributing data without checking the terms.

## Final packaging

Include the working BI file, readable screenshots, code, a data-quality report, KPI dictionary, model diagram, recommendation table and executive summary. A live dashboard link is optional: reviewability should not depend on a recruiter signing in.

### Completion check

A person following the README can reproduce the analytics from the source download without relying on your local absolute paths or private credentials.

## 15 / Build sequence and final checks

A suggested six-week sequence at roughly 10–15 focused hours per week. This is an effort estimate, not a deadline guarantee. Complete the phase checks before adding optional features.

| WHEN   | WORK                                                                                        | REVIEWABLE OUTPUT                                                  |
|--------|---------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| Week 1 | Learn SQL/pandas foundations; settle BI access; define business scope; ingest/profile data. | Business brief, environment, source manifest and dictionary.       |
| Week 2 | Clean and validate; document exceptions; build facts and dimensions.                        | ETL scripts, quality report and reconciled model.                  |
| Week 3 | Build sales, customer and operations SQL; plot EDA.                                         | Query modules, KPI dictionary and first measured findings.         |
| Week 4 | Build RFM, mature cohorts and 90-day repeat; compare reviews by delivery group.             | Customer marts, heatmap, statistical report and opportunity table. |
| Week 5 | Create Power BI measures/pages; test filters and compare SQL totals.                        | Five-page dashboard, measure documentation and screenshots.        |
| Week 6 | Write recommendations and scenarios; rerun pipeline; finish README and summary.             | Reproducible portfolio and one-page executive summary.             |

### Final acceptance checklist

- Business questions, status rules and observation cutoff are explicit.
- All source files have provenance; key and monetary checks pass or exceptions are documented.
- Facts preserve grain; customer analysis uses the correct stable identity.
- SQL and notebooks reproduce; cohort cells respect maturity; statistics include uncertainty.
- Power BI measures reconcile under supported filters; no duplicate sales or ratings.
- Recommendations use actual findings; estimates show assumptions and BRL units.
- No unsupported profit, funnel, churn, causal or realized-impact claim appears.
- README, screenshots, model diagram, BI report and summary are present; credentials are excluded.

### Be ready to explain the project in an interview

Practice answering: What decision did your analysis support? Why did you choose each fact grain? How did you prevent row multiplication? How did you handle tied RFM scores and incomplete cohorts? Why is the delivery/review finding observational? Which recommendation would you test first, and how would you measure success?

**Resume rule:** add verified row counts and real deliverables after completion. Describe what you analyzed and built; report estimated opportunity as estimated, never as achieved revenue.

## 16 / Sources and learning resources

Primary references checked on 6 October 2026. The implementation sequence, thresholds and effort estimates in this guide are proposed project design choices. The dataset must be downloaded and analyzed before actual findings can be written.

### [1] Olist dataset and metadata

Download source, dataset coverage, table descriptions and customer identifiers. Review its license before redistribution.

<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

### [2] Microsoft: Power BI Desktop

Installation and Windows requirements. Settle your Power BI access before the dashboard phase.

<https://learn.microsoft.com/en-us/power-bi/fundamentals/desktop-get-the-desktop>

### [3] Microsoft: star schema guidance

Fact/dimension roles, grain, relationships and date modeling.

<https://learn.microsoft.com/en-us/power-bi/guidance/star-schema>

### [4] PostgreSQL: window functions

Learn ranking, partitioning and running or previous-period calculations.

<https://www.postgresql.org/docs/current/tutorial-window.html>

### [5] pandas: merging and joining

Study join behavior and merge validation to prevent unintended row multiplication.

[https://pandas.pydata.org/docs/user\\_guide/merging.html](https://pandas.pydata.org/docs/user_guide/merging.html)

### [6] SciPy: Mann-Whitney U

Read assumptions, alternatives and cautions before using the rating comparison test.

<https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.mannwhitneyu.html>

## How to learn without delaying the build

Use a small sample to practice a technique, then apply it to the project immediately. For SQL, start with joins and GROUP BY before windows. For pandas, focus on dtypes, dates, groupby and merge validation [5]. For BI, understand relationships before decorating pages. Keep optional ML out of the critical build path.

## Your first working session

Create the repository, write the business brief, confirm Windows access for Power BI, download the dataset, and profile orders/customers/items. End the session with a source manifest and a correct order-level merchandise aggregate.